

Namn
Anna Backolars. Ali Adrian Nasrat

DU-identitet
H22alian, h23annba

Kurskod
GIK2NV

Termin och år
HT2025

1. Projektbeskrivning

Vi har valt att skapa en resedatabas där distans, städer och flyg är noder. Grafdatabaser lämpar sig för detta eftersom relationerna mellan flyg och städer är många-till-många och kan visualiseras effektivt.

1.1 Grafdatabas

Grafdatabaser är särskilt lämpliga för detta projekt eftersom de hanterar många-till-många relationer effektivt. En stad kan ha flera flyg som avgår till olika destinationer och ett flyg kan koppla samman flera städer via mellanlandningar. I en traditionell relationsdatabas skulle detta kräva flera tabeller och komplexa JSON-satser, medan Neo4j gör det möjligt att snabbt navigera mellan noder och se hur städer är sammankopplade via flyg och distanser.

Genom att använda Neo4j kan vi enkelt ställa frågor som:

1. Vilka städer kan man nå direkt från en viss stad?
2. Vilka rutter kräver mellanlandning?
3. Vilken stad har flest flygförbindelser?
4. Vilka städer har det kortaste avståndet mellan varandra?

Detta gör grafdatabaser användbara för att modellera rese- och transportnätverk, eftersom fokus ligger på relationerna mellan noderna snarare än på statiska tabeller.

1.2 Datamodell för grafdatabasen

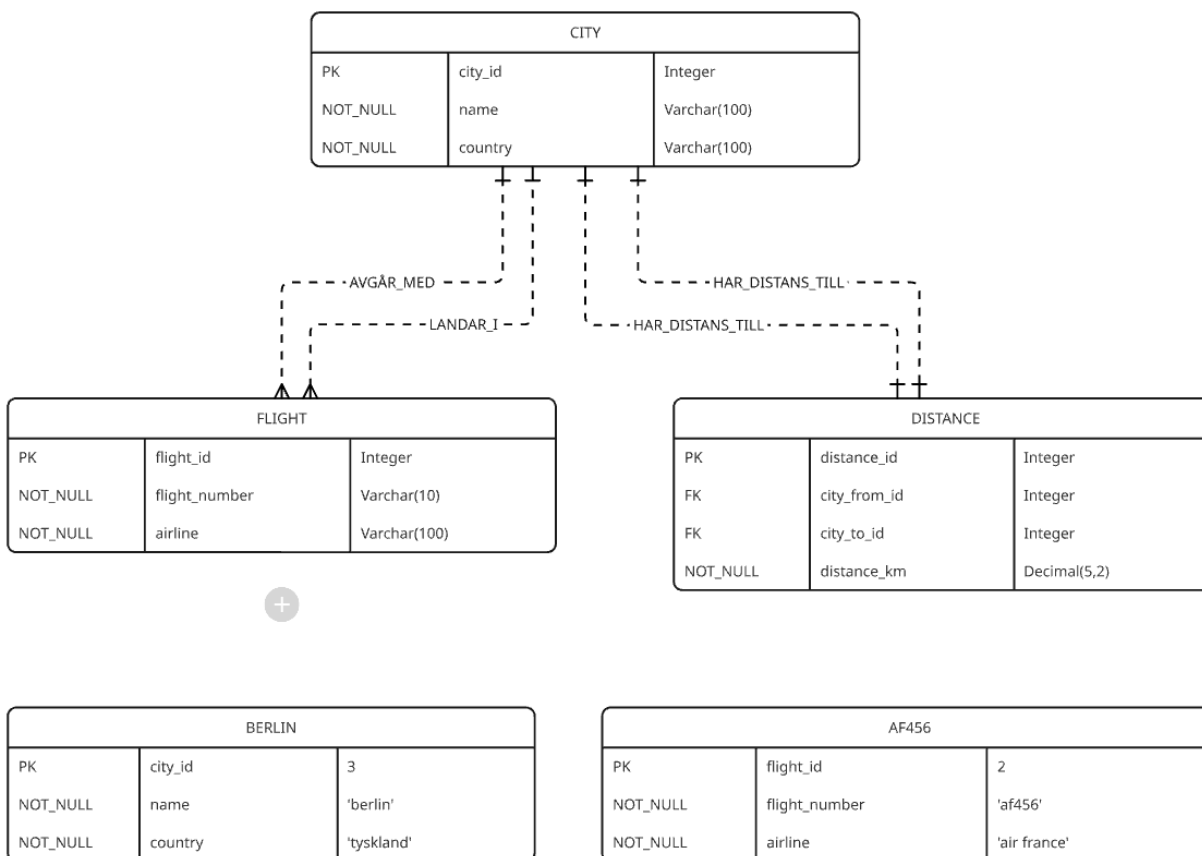
Figuren nedan visar den konceptuella modellen för projektets grafdatabas i Neo4j. Modellen består av tre centrala noder.

- CITY representerar en stad och innehåller egenskaperna name och country.
- FLIGHT representerar ett flyg med egenskaperna flight_number och airline.
- DISTANCE representerar kopplingen mellan två städer och innehåller egenskaperna distance_km.

Relationerna AVGÅR_MED, LANDAR_I och HAR_DISTANS_TILL visar hur noderna är sammankopplade. En stad kan ha flera avgående flyg (AVGÅR_MED), varje flyg kan landa i en eller flera städer (LANDAR_I) och städer är förbundna med varandra genom distansrelationer (HAR_DISTANS_TILL).

De två tabellerna nedanför grafstrukturen illustrerar på enkilda instanser av noderna i databasen:

- BERLIN representerar en specifik stad med city_id = 3, name = "berlin" och country = "tyskland"
- AF456 representerar ett specifikt flyg med flight_id = 2, flight_number = "af456" och airline = "air france"



1.3 Neo4j

Steg 1 – Constraints

I detta steg skapades unika constraints för noderna Stad och Flyg. Dessa säkerställer att varje stad och varje flyg har ett unikt namn respektive flygnummer, vilket förhindrar dubletter.

```
1 CREATE CONSTRAINT stad_namn_unik IF NOT EXISTS
2 FOR (s:Stad) REQUIRE s.namn IS UNIQUE;
3
4 CREATE CONSTRAINT flyg_nr_unik IF NOT EXISTS
5 FOR (f:Flyg) REQUIRE f.flygnummer IS UNIQUE;
```

```
neo4j$ CREATE CONSTRAINT stad_namn_unik IF NOT EXISTS FOR (s:Stad) REQUI... ✓
neo4j$ CREATE CONSTRAINT flyg_nr_unik IF NOT EXISTS FOR (f:Flyg) REQUIRE... ✓
```

Unika constraints skapade för noderna Stad och Flyg.

Steg 2 – Skapa städer

Här skapades fyra städer Stockholm, Köpenhamn, Oslo och Berlin var och en med namn och tillhörande land. Dessa noder utgör grunden för grafstrukturen och fungerar som start- och destinationspunkter i modellen.

```
1 CREATE CONSTRAINT stad_namn_unik IF NOT EXISTS
2 FOR (s:Stad) REQUIRE s.namn IS UNIQUE;
3
4 CREATE CONSTRAINT flyg_nr_unik IF NOT EXISTS
5 FOR (f:Flyg) REQUIRE f.flygnummer IS UNIQUE;
6
7 CREATE (:Stad {namn:'Stockholm', land:'Sverige'});
8 CREATE (:Stad {namn:'Berlin', land:'Tyskland'});
9 CREATE (:Stad {namn:'Köpenhamn', land:'Danmark'});
10 CREATE (:Stad {namn:'Oslo', land:'Norge'});
```

```
neo4j$ CREATE CONSTRAINT stad_namn_unik IF NOT EXISTS FOR (s:Stad) REQUIRE s.namn IS UNIQUE ✓
neo4j$ CREATE CONSTRAINT flyg_nr_unik IF NOT EXISTS FOR (f:Flyg) REQUIRE f.flygnummer IS UNIQUE ✓
neo4j$ CREATE (:Stad {namn:'Stockholm', land:'Sverige'}) ✓
neo4j$ CREATE (:Stad {namn:'Berlin', land:'Tyskland'}) ✓
neo4j$ CREATE (:Stad {namn:'Köpenhamn', land:'Danmark'}) ✓
neo4j$ CREATE (:Stad {namn:'Oslo', land:'Norge'}) ✓
```

Fyra städer skapade som noder i databasen.

Steg 3 – Distans

Distanserna mellan städerna lades in som relationer (HAR_DISTANS_TILL) med egenskapen avstånd_km. Detta gör det möjligt att analysera hur städerna är kopplade geografiskt samt att till exempel kunna beräkna den kortaste ruten mellan städerna.

```
1 WITH [
2   ['Stockholm', 'Köpenhamn', 522],
3   ['Köpenhamn', 'Berlin', 355],
4   ['Stockholm', 'Oslo', 416],
5   ['Oslo', 'Köpenhamn', 484],
6   ['Berlin', 'Stockholm', 810]
7 ] AS dists
8 UNWIND dists AS d
9 MATCH (a:Stad {namn:d[0]}), (b:Stad {namn:d[1]})
10 MERGE (a)-[:HAR_DISTANS_TILL {avstånd_km:d[2]}]-(b)
11 MERGE (b)-[:HAR_DISTANS_TILL {avstånd_km:d[2]}]-(a);
```

Table

Set 10 properties, created 10 relationships, completed after 8 ms.

Relationer (HAR_DISTANS_TILL) mellan städer med avstånd i km.

Steg 4 – Flyg och relationer

Tre flyg skapades med bolagen som SAS, Air France och Norwegian. Relationerna AVGÅR_MED och LANDAR_I kopplar varje flyg till sina respektive avrese- och destinationsstäder.

```
1 CREATE (:Flyg {flygnummer:'SK123', bolag:'SAS'});
2 CREATE (:Flyg {flygnummer:'AF456', bolag:'Air France'});
3 CREATE (:Flyg {flygnummer:'D789', bolag:'Norwegian'});
4
5 MATCH (sto:Stad {namn:'Stockholm'}),
6       (ber:Stad {namn:'Berlin'}),
7       (cph:Stad {namn:'Köpenhamn'}),
8       (osl:Stad {namn:'Oslo'});
9 MATCH (f1:Flyg {flygnummer:'SK123'}),
10        (f2:Flyg {flygnummer:'AF456'}),
11        (f3:Flyg {flygnummer:'D789'});
12
```

```
neo4j$ CREATE (:Flyg {flygnummer:'SK123', bolag:'SAS'})
neo4j$ CREATE (:Flyg {flygnummer:'AF456', bolag:'Air France'})
neo4j$ CREATE (:Flyg {flygnummer:'D789', bolag:'Norwegian'})
neo4j$ MATCH (sto:Stad {namn:'Stockholm'}), (ber:Stad {namn:'Berlin'}), (cph:Stad {namn:'Köpenhamn'}), (osl:Stad {namn:'Oslo'}) MATCH (f1:Flyg {flygnummer:'SK123'}), (f2:Flyg {flygnummer:'AF456'}), (f3:Flyg {flygnummer:'D789'})
```

Flygnoder skapade och kopplade till städer genom AVGÅR_MED och LANDAR_I.

Verifiera att alla noder och relationer finns

Körningen bekräftar att modellen innehåller 4 städer och 3 flyg noder, vilket överensstämmer med den planerade datamodellen.

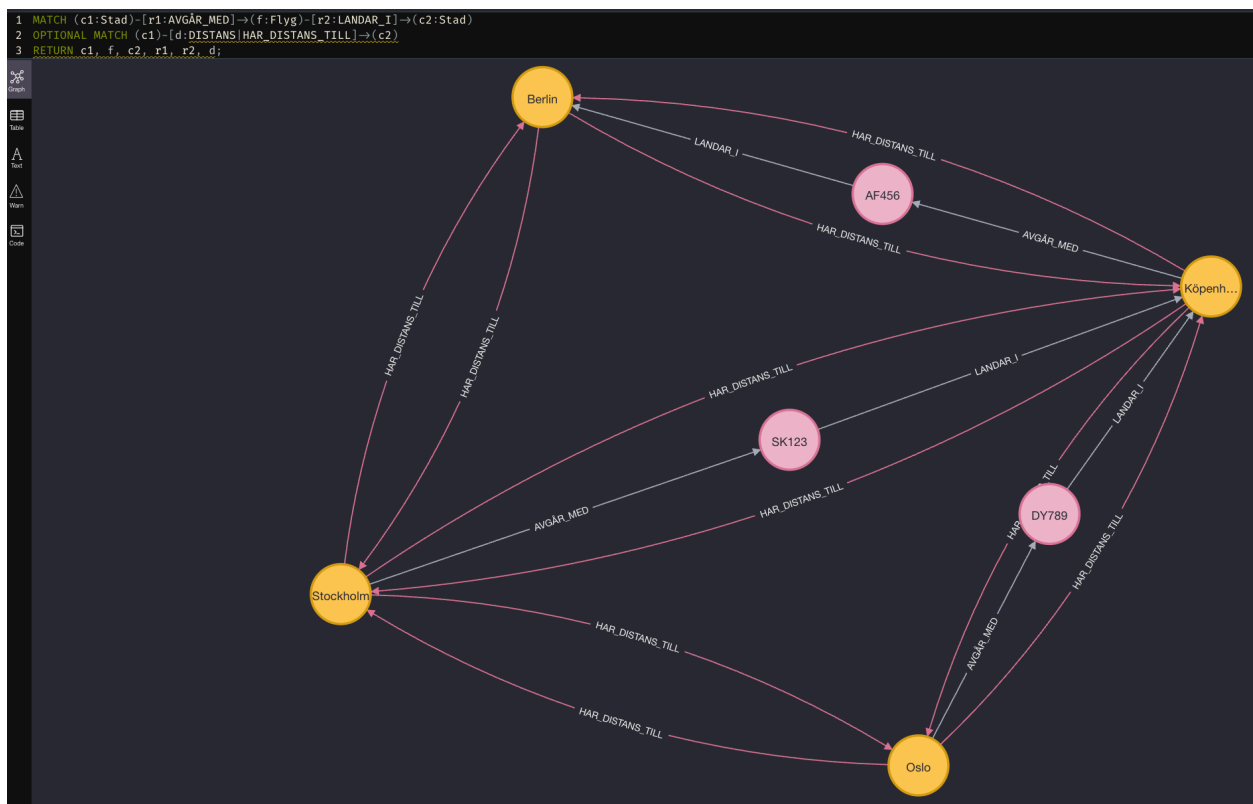
```
neo4j$ MATCH (n) RETURN labels(n) AS label, count(*) AS antal ORDER BY antal DESC;
```

label	antal
["Stad"]	4
["Flyg"]	3

Sammanställning av antal noder per label efter implementation.

Visa alla flyg mellan städer (grafvy)

Den grafiska visualiseringen visar hur städer och flyg hänger samman. Gula noder representerar städer, rosa noder flyg och pilarna anger riktningarna för AVGÅR_MED och LANDAR_I. Grafen ger en intuitiv överblick över hela flygnätverket.



Grafisk visualisering av alla flygförbindelser mellan städer.

Q1 – Direkta destinationer från viss stad

Frågan visar vilka städer som kan nå direkt från Stockholm utan mellanlandning. Resultatet visar att Köpenhamn är en direkt destination från Stockholm.

```
1 WITH 'Stockholm' AS start
2 MATCH (s:Stad {namn:start})-[:AVGÅR_MED]→(:Flyg)-[:LANDAR_I]→(dest:Stad)
3 RETURN start AS från, dest.namn AS direkt_destination
4 ORDER BY direkt_destination;
```

från	direkt_destination
"Stockholm"	"Köpenhamn"

Direkta destinationer från Stockholm utan mellanlandning.

Q2 – Flyg som kräver mellanlandning

Här kontrollerades om en resa från till exempel Stockholm till Berlin kräver mellanlandning. Resultatet visar att det inte finns ett direktflyg, vilket innebär att minst en mellanlandning behövs.

```
1 WITH 'Stockholm' AS start, 'Berlin' AS mål
2 OPTIONAL MATCH (a:Stad {namn:start})-[:AVGÅR_MED]→(:Flyg)-[:LANDAR_I]→(b:Stad {namn:mål})
3 WITH start, mål, COUNT(b) AS direkt
4 MATCH (s:Stad {namn:start})-[:AVGÅR_MED]→(:Flyg)-[:LANDAR_I]→(:Stad)-[:AVGÅR_MED*0..2]→(:Flyg)-[:LANDAR_I]→(t:Stad {namn:mål})
5 WHERE direkt = 0
6 RETURN start, mål, 'Kräver mellanlandning' AS status
7 LIMIT 1;
```

start	mål	status
"Stockholm"	"Berlin"	"Kräver mellanlandning"

Resultat som visar att resan från Stockholm till Berlin kräver mellanlandning.

Q3 – Flygförbindelser

Frågan identifierar städer med flest flygförbindelser. Resultatet visar att Köpenhamn är den mest centrala knutpunkten i vårt nätverk.

```
1 MATCH (s:Stad)
2 OPTIONAL MATCH (s)-[:AVGÅR_MED]→(:Flyg) WITH s, count(*) AS ut
3 OPTIONAL MATCH (:Flyg)-[:LANDAR_I]→(s) WITH s, ut, count(*) AS in
4 RETURN s.namn AS stad, (ut + in) AS total_flyg
5 ORDER BY total_flyg DESC, stad ASC
6 LIMIT 5;
```

stad	total_flyg
"Köpenhamn"	3
"Berlin"	2
"Oslo"	2
"Stockholm"	2

Städer rangordnade efter flygförbindelser.

Q4 – Kortaste avståndet mellan två städer

Frågan jämför alla HAR_DISTANS_TILL-relationer och returnerar det kortaste avståndet i kilometer. Resultatet visar att det kortaste avståndet är mellan Köpenhamn och Berlin (355 km).

```
1 MATCH (a:Stad)-[r:HAR_DISTANS_TILL]→(b:Stad)
2 WITH a, b, r.avstånd_km AS dist
3 ORDER BY dist ASC
4 LIMIT 1
5 RETURN
6 a.namn AS Stad1,
7 b.namn AS Stad2,
8 dist AS Kortaste_Avstånd_km;
```

Stad1	Stad2	Kortaste_Avstånd_km
"Berlin"	"Köpenhamn"	355

Kortaste avståndet mellan två städer.

1.4 Sammanfattning

Projektet resulterade i en fungerande grafdatabas i Neo4j som modellerar flygförbindelser mellan städer på ett överblickbart sätt. Genom att representera städer, flyg och distanser som noder och relationer kunde komplexa samband visualiseras och analyseras enkelt. Databasen klarade samtliga frågeställningar, inklusive direkta destinationer, mellanlandningar och flygförbindelser vilket bekräftar att den implementerade modellen fungerar som tänkt. Arbetet visar tydligt hur grafdatabaser lämpar sig för att hantera många-till-många relationer och ge snabba insikter i nätverksbaserade system som transport- och reseplanering.